1 Problem Description

Consider an Alloy model M, with a command C, of the form run/check P for scope s. When fed into the Analyzer, there are 4 possible outcomes:

- Instance found - this occurs when C is a run command, and the constraints of M and P are satisfied by an instance I, which is shown by the Analyzer.
- No Instances found - this occurs when when C is a run command, and there is no instance I capable of satisfying the constraints of M and P. This includes the case where the constraints of just M are unsatisfiable.
- Counterexample found - this occurs when C is a check command, and there is an instance I which satisfies the constraints of M, and violates the constraints of P.
- No Counterexample found - this occurs when C is a check command, and every instance I which satisfies the constraints of M, also satisfies the constraints of P. This includes the case where the constraints of just M are unsatisfiable.

The goal of a translation to TLA+ is that when the translated model is run using TLC, the output produced by TLC can be used to infer the output the Analyzer would have produced, had the original model been run on the Analyzer. A translation scheme needs to map from the output of TLC to the output of the Analyzer.

A translation is correct, if and only if the Analyzer's outputs correspond to the outputs produced by TLC. A translation is efficient, if the time taken to run a translated model using TLC is lower than the time taken to run the original model using the Analyzer.

1.1 Run-Check duality

Consider a model M with a command C, where C is of the form check P for scope s. Consider C', which is of the form run $\neg$P for scope s.

Claim: C and C' produce the same set of instances.

Proof:

Sub-claim: any instance produced by C is also produced by C'

- Let I be an instance of the model, produced by C.
- Since C is a check command, I satisfies M, but not P.
- Since I does not satisfy P, I satisfies $\neg$P.
- Since I satisfies $\neg$P and M, I is produced by run $\neg$P (which is what C' is)
- Thus, any instance produced by C is also produced by C'

Sub-claim: any instance produced by C' is also produced by C

- Let I be an instance of the model, produced by C'.
- Since C is a run command, I satisfies M and P.
- Since I satisfies P, I does not satisfy P.
- Since I does not satisfy P but satisfies M, I is produced by check P (which is what C is)
- Thus, any instance produced by C' is also produced by C

Final proof:

- Let S be the set of all instances produced by C, and S' be the set of all instances produced by C'
- The claim is that S = S'
- From the first result, S is a subset of S'
- From the second result, S' is a subset of S
- Since S and S' are subsets of each other, they are the same set

This means that before translation, any command can be turned into an equivalent run command, or an equivalent check command, by inverting the property P within that command. Thus, a translation scheme that supports either run commands or check commands, is sufficient to support both types of commands.

2 TLC's behavior

The following is a guess at the model-checking algorithm used by TLC, presented at a high level of abstraction. Since the translator targets TLC, the translator is designed under the assumption that translated model is subject to an algorithm similar to the one presented below:

- all_states - this is a hash table that stores a collection of states. There are two operations performed - adding a state to the table, and looking a state up on the table to see if it already exists.
- state_queue - this is a queue that stores a collection of states. There are three operations performed - pushing a state to the tail of the queue, popping a state from the head of the queue, and looking up the length of the queue.
- Init_MC - this a function that returns all the states that satisfy the membership constraints of the Init formula.

- Init_LC(s) - this a function that returns true iff the state s satisfies the logical constraints of the Init formula.
- Next_MC(s) - this a function that returns all the states that satisfy the membership constraints of the Next formula, where the unprimed variables belong to s, and the primed variables belong to the states in the returned set.
- Next_LC(s,s') - this is a function that returns true iff the states s and s' satisfy the logical constraints of the Next formula, where the primed variables belong to s' and the unprimed variables belong to s.
- exit() - this represents a termination of the algorithm.
- Inv(s) - this is a function that returns true iff s satisfies every invariant in the TLA specification.
- write(s) - this represents writing the state s into the system's memory, which persists after running TLC.
- EXIT_ON_DEADLOCK - this represents a flag set when TLC is run, specifying how to behave if TLC detects a deadlock - i.e. a state from which no transitions can be taken.
- The algorithm has two phases - in the first phase, TLC computes all valid initial states, and writes them to memory. In the second phase, TLC uses the initial states to generate and check more states, by taking valid transitions. The algorithm terminates when no more unseen states remain.

Since the process of checking whether some state s in in all_states is done via comparing the hashes, the is a small probability that TLC did not explore all possible states. This probability is reported upon a successful exit.

3 Translation schemes

Algorithm 1 Model-checking with TLC

```
1: all_states ← {}
2: state_queue ← []
3: for s in Init_MC() do
4:    if Init_LC(s) then
5:       if not Inv(s) then
6:          exit("invariant violation")
7:       end if
8:       all_states.add(s)
9:       state_queue.push(s)
10:      write(s)
11:   end if
12: end for
13: while state_queue ≠ [] do
14:   s ← state_queue.pop()
15:   S_nexts ← Next_MC(s)
16:   SOME_NEXT_STATE ← false
17:   for s' in S_nexts do
18:      if Next_LC(s, s') then
19:         if not s' in all_states then
20:            if not inv(s') then
21:               write(s')
22:               exit("invariant violation")
23:            else
24:               all_states.add(s')
25:               state_queue.push(s')
26:               write(s')
27:            end if
28:         end if
29:         SOME_NEXT_STATE ← true
30:      end if
31:   end for
32:   if not SOME_NEXT_STATE and EXIT_ON_DEADLOCK then
33:      exit("deadlock detected")
34:   end if
35: end while
36: exit("success")
```